Princess Sumaya University for Technology
جامعة الأميرة سميّة للتكنولوجيا

Computer Science Department

Second Exam

CS-11313: Algorithm Analysis and Design

Spring Semester 2022/2023
Sunday, 7-March-23 @ 4:15 PM

Student Name:		**Student Number:**
Instructor:	**Section # (or time):**	**Signature:**

Instructions

- Time allowed: **60 Minutes**
- This booklet should be 7 pages long, including this page. If not, report this to the instructor.
- Answer all questions in the examination paper provided.
- Write your answers clearly: you may not receive full credit if your answer is unreadable.

Question	Max Mark	Your Mark	CLO
1	**32**		CLO 1
2	**30**		CLO 2
3	**38**		CLO 3
Total	**100**		

Question 1: Recurrences [32 Points]

(a) [18 points] Solve the following three recurrences using the **Master Theorem** if possible. Show ***all*** steps for full credit. If it is not possible to solve the problem using the Master Theorem, state so and explain why.

1. **T(n) = 27T(n/3) + 3 n^3**

f(n)= 3 n^3 **Number of Leaves=** $n^{\log_3^{27}}$**=**n^3

$$\theta(f(n)) = number\ leaves$$

$$\theta(n^3) = n^3$$

➔ **Case 2: answer will be** $= \theta(f(n)\ log\ n) = \ \theta(n^3 log\ n)$

2. **T (n) = 9T (n/3) + n^2/ log n**

f(n)= n^2/ log n **L=Number of Leaves=** $n^{\log_3^9}$**=**n^2

$$\theta(n^3) = n^3$$

Solution 1:

➔ **Since they are not equivalent, check if there is a polynomial difference**

$$f(n)/\ L = \ (n^2/log\ n)/(n^2) = 1/log\ n$$

There is no polynomial difference between f(n) and number of leaves
Master method cannot be used.

Solution 2:
Since f(n) is smaller, we need to try Case 1

➔ **Try case 1:** $= f(n) = \ O(n^{\log_{3-\varepsilon}^9}) = \ (n^2/\log\ n) = O(n^{2-\varepsilon})$
There is $\varepsilon > 0\ that\ can\ keep\ this\ equality\ correct$
Master method cannot be used.

3. **T (n) = 2T (n/4) + $n^{0.499}$**

f(n)= $n^{0.499}$ **Number of Leaves=** $n^{\log_4^2}$**=**$n^{0.5}$
Since f(n) is smaller, we need to try Case 1

➔ **Try case 1:** $= f(n) = \ O(n^{\log_{4-\varepsilon}^2}) = \ (n^{0.499}/) = O(n^{0.5-\varepsilon})$
➔ $\varepsilon \leq 0.001$ **will make case 1 applicable.**
➔ **Answer:**

$$\theta(n^{0.5})\ or\ \theta(\sqrt{n})$$

(b) [14 points] Solve the following recurrence using the **Iterative substitution method**.

$T(n) = n^2 + T(n-1)$, where $T(0) = 1$

$T(n) = n^2 + T(n-1)$
$= n^2 + (n-1)^2 + T(n-2)$
$= n^2 + (n-1)^2 + (n-2)^2 + T(n-3)$
$= n^2 + (n-1)^2 + (n-2)^2 + (n-3)^2 + T(n-4)$
$= n^2 + (n-1)^2 + (n-2)^2 + (n-3)^2 + \ldots + (n-(k-1))^2 + T(n-k)$

Will stop when $T(0)=1$ ➔ $T(n-k)=1$ when $k=n$, so will stop when $k=n$
➔ substitute $k=n$
$= n^2 + (n-1)^2 + (n-2)^2 + (n-3)^2 + \ldots + (n-(n-1))^2 + T(0)$

$= n^2 + (n-1)^2 + (n-2)^2 + (n-3)^2 + \ldots + (1)^2 + 1$

$$1 + \sum_{0}^{n} i^2$$

=use summation formula

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + 3^2 + \ldots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

answer= $1+n(n+1)(2n+1)/6 = \theta(n^3)$

Question 2: Greedy Algorithms [30 points]

You are the head of the photography club at PSUT. The *Open Day* is approaching and you are planning for it. You have received a number of new cameras and are planning to use them to provide media coverage. The schedule of the Open Day includes events organized by different clubs, where each event has a specified start time and end time, and different events can happen at the same time.

For each of the following scenarios, mention if the problem can be solved using a greedy strategy or not. If it can, describe the greedy strategy with <u>less than 15 words</u>.

Each of these problems corresponds to a problem we covered in class.

1. Some club members want to minimize the number of used cameras (to reduce wear and tear).

   Problem: Find the minimum number of cameras needed to cover all the events (a camera can cover only 1 event at a time).

   Greedy Strategy: Assign a camera to the earliest starting event first.

   ☐ *Cannot* be solved with a greedy algorithm

   ■ *Can* be solved with a greedy algorithm.

2. Some club members insist that you don't use more than one camera. To avoid looking bad, they want to cover the maximum number of events possible.

   Problem: Find the maximum number of events that can be covered using a single camera (a camera can cover only 1 event at a time).

   Greedy Strategy: Cover the earliest finishing event first. This is the Activity Selection problem we covered in class.

   ☐ *Cannot* be solved with a greedy algorithm

   ■ Can be solved with a greedy algorithm.

3. Some club members argued that you should maximize the profit you can get from the Open Day.

   Problem: Given a charge that every club is willing to pay to cover their event, find the maximum amount of money that you can get if you use a single camera (a camera can cover only 1 event at a time)

   Greedy Strategy: This is the Weighted Activity Selection Problem we solved using Dynamic Programming

   ■ Cannot be solved with a greedy algorithm

   ☐ Can be solved with a greedy algorithm.

4. A club member noted he can borrow from a relative (for free) enough cameras to cover all of the events. However, he noted that the club's cloud storage subscription allows uploading at most 20 GBs.

Problem: Given how many MBs/GBs of video storage each event requires, Find the maximum amount of money that you can get from the Open Day such that the total amount of video storage does not exceed 20 GBs (assuming that every event can get its own camera and that you will cover the whole duration of every event you decide to cover).

- [x] Cannot be solved with a greedy algorithm
- [ ] Can be solved with a greedy algorithm.

Greedy Strategy: This is the 0-1 Knapsack problem we solved using Dynamic Programming

5. A club member suggested that it is OK if an event is not covered for its full duration. In this case, the club would charge for only the duration covered.

Problem: Find the maximum amount of money that you can get from the Open Day, such that the total video recording storage does not exceed 20 GBs (assuming that an event can be covered partially and that there are enough cameras to cover every event).

- [ ] Cannot be solved with a greedy algorithm
- [ ] Can be solved with a greedy algorithm.

Greedy Strategy: Take the event whose charge/time is maximum. This is the Fractional Knapsack problem we covered in class.

Question 3: Dynamic Programming Algorithms [38 Points]

A. [8 points] Consider the following optimal structure for the *collecting apples* problem we studied in class.

max_apples(i, j) = apples[i][j] + MAX(**max_apples**(i-1, j),
max_apples(i, j-1))

Write a new recurrence equation for the optimal structure to reflect a new version of the problem. The new version has the following changes:

1. Your goal is to collect the **least** number of apples from the top-left corner to the bottom-right corner in the grid.
2. You have 3 types of moves: **Down**, **Right**, **TwoStepsRight**.
   TwoStepsRight allows moving 2 cells to the right (jumping over the right cell without taking the apples).

Write your answer in the box below.

min_apples(i, j) = apples[i][j] +Min(

Min(**min_apples**(i-1, j), **min_apples**(i, j-1)), **min_apples**(i, j-2))

B. [14 points] Use bottom-up Dynamic Programming to compute an optimal solution to the following instance of the problem:

Knapsack capacity = 6 Kg
Item 1: weight = 2 Kg, value = $8
Item 2: weight = 1 Kg, value = $5
Item 3: weight = 4 Kg, value = $15

B1. [10 points] Show the complete dynamic programming table, then clearly indicate which items will be taken into the knapsack to obtain the maximum value.

Item\Weight	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	8	8	8	8	8
2	0	5	8	13	13	13	13
3	0	5	8	13	15	20	23

Items that will be taken: Item 1 and Item 3

B2. [2 points] If possible, use the same dynamic programming table to calculate the maximum we will get if we only have items 1 and 2 with a knapsack of capacity 4. If this subproblem has no corresponding cell in the table, write: “The subproblem is not in the table”.

13

B3. [2 points] If possible, use the same dynamic programming table to calculate the maximum we will get if we only have items 1 and 3 with a knapsack of capacity 5. If this subproblem has no corresponding cell in the table, write: “The subproblem is not in the table”.

The subproblem is not in the table

C. [16 points] Consider a new problem known as "**Subset Sum**". Given an array data[] of **n** strictly positive integers, and an integer **k**, the goal is to return the number of subsets in the data[] array that sum to exactly **k.**

Consider the following recursive definition for the optimal solution **OPT**(k,i) of the problem. This represents the optimal substructure of the problem.

OPT(k,i)	= 1	**if** k = 0
	= 0	**if** k < 0 **or if** i=n
	= **OPT**(k-data[i], i+1) + **OPT**(k, i+1)	otherwise

Convert the above recursive definition of **OPT**(k, i) to a <u>bottom-up</u> dynamic programming implementation.

Note. You can implement a <u>memoized</u> solution instead. However, you will receive up to 10/16 points only.

Memoized Solution

```
OPT(data[], k):
n=length(data)
Initialize array result[n+1][k+1] to -1
OPT(data[], k, 0, result[],n)
Return result[0][k]


 OPT(data[], k, i, result[],n):

    if i == n or k < 0:
        return 0
    if k == 0:
        return 1
    if result[i, k] !=-1
        return result[i, k]
    result[i, k]= OPT(data, k-data[i], i+1, result,n) + OPT(data, k, i+1, result,n)
    return result[i, k]
```

Bottom-up

```
 OPT2(data[], k) {

       n=length(data)
       create a 2D array result[n][k+1]

       //set column 0 to 1

       result[][0] = 1

       //data[n-1] will return 1 only if it is value equal k

      for int j = 0 to k
             if (data[n - 1] == j)
                    result[n-1][j] = 1
             else
                    result[n-1][j] =0



       for i = n-2 To 0
             for j = 0 To k

                    if (j > data[i])
                      result[i][j] = result[i + 1][j - data[i]] + result[i + 1][j]
                    else if (j == data[i])
                      result[i][j] = 1
                    else
                      result[i][j] = result[i + 1][j]

       return result[0][k];

}
```